

Lab6. Oracle object-relational database

Content

- [About Oracle object](#)
- [Object types](#)
- [References](#)
- [More reading](#)
- [Exercises](#)

About Oracle Object

Oracle object types are user-defined types that make it possible to [model real-world entities](#) such as customers and purchase orders as objects in the database. Oracle object technology is a [layer of abstraction](#) built on Oracle relational technology. New object types can be created from any built-in database types and any previously created object types, object references, and collection types.

Advantages of objects:

- Objects can encapsulate operations along with data.
- Objects are efficient.
- Objects can represent part-whole relationships.

Object types

An object type is a kind of [datatype](#). We can use it in the same ways that we use more familiar datatypes such as NUMBER or VARCHAR2. For example, you can specify an object type as the datatype of a column in a relational table, and you can declare variables of an object type. We use a variable of an object type to contain a value of that object type. A value of an object type is an instance of that type. An [object instance](#) is also called an object.

Example 1 shows how to create an object type named person_typ. In the example, an object specification and object body are defined. For information on the CREATE TYPE SQL statement, see Oracle Database SQL Reference. For information on the CREATE TYPE BODY SQL statement, see Oracle Database SQL Reference.

Example 1: Creating the person_typ Object.

```
CREATE TYPE person_typ AS OBJECT (  
    idno          NUMBER,  
    first_name    VARCHAR2(20),  
    last_name     VARCHAR2(25),  
    email         VARCHAR2(25),  
    phone         VARCHAR2(20),  
    MEMBER FUNCTION get_idno RETURN NUMBER,  
    MEMBER PROCEDURE disp_details (SELF IN OUT NOCOPY person_typ));  
/
```

```

CREATE TYPE BODY person_typ AS
  MEMBER FUNCTION get_idno RETURN NUMBER IS
  BEGIN
    RETURN idno;
  END;

  MEMBER PROCEDURE disp_details (SELF IN OUT NOCOPY person_typ) IS
  BEGIN
    -- use PUT_LINE procedure of the DBMS_OUTPUT package to display details
    DBMS_OUTPUT.PUT_LINE(TO_CHAR(idno) || ' ' || first_name
                          || ' ' || last_name);
    DBMS_OUTPUT.PUT_LINE(email || ' ' || phone);
  END;
END;

```

Object types have some [important differences](#) from the more familiar datatypes that are native to a relational database:

- A set of object types does [not come ready-made](#) with the database. Instead, we define the specific object types we want by extending built-in types with user-defined ones, as shown in Example 1.
- Object types are composed of parts called attributes and methods.

[Objects](#)

When we create an object type variable, we create an instance of the type, and the result is an object. An object has the attributes and methods defined for its type. Because an object instance is concrete, we can assign values to its attributes and call its methods.

We use the `CREATE TYPE` statement to define object types. Defining an object type does not allocate any storage. After they are defined, object types can be used in SQL statements, and in most of the same places, we can use types like `NUMBER` or `VARCHAR2`.

The following example defines a relational table to keep track of the contacts.

Example 2: Creating the contacts table with an object type column.

```

CREATE TABLE contacts (
  contact          person_typ,
  contact_date     DATE );

INSERT INTO contacts VALUES (
  person_typ (65, 'Verna', 'Mills', 'oop@gmail.com', '0912345678'),
  '24 Jun 2003'
);

```

Object Methods

Methods are functions or procedures we can declare in an object type definition to implement behavior that we want objects of that type to perform. For example, a method is declared in Example 1 to allow comparisons between `person_typ` objects.

A principal use of methods is to provide [access to an object's data](#). You can define methods for operations that an application will likely want to perform on the data so that the application does not have to code these operations. To operate, an application calls the appropriate method on the appropriate object.

The following example uses the `get_idno()` method to display the ID number of persons in the `contacts` table.

Example 3: Using the `get_idno` object method.

```
SELECT c.contact.get_idno() FROM contacts c;
```

Type Evolution

Using an `ALTER TYPE` statement, we can modify or evolve an existing object type to make the following changes:

- Add and drop attributes
- Add and drop methods
- Modify a numeric attribute to increase its length, precision, or scale
- Modify a varying length character attribute to increase its length
- Change a type's `FINAL` and `INSTANTIABLE` properties

Dependencies of a type to be altered are checked using the same validations applied for a `CREATE TYPE` statement. If a type or any of its dependent types fails the type validations, the `ALTER TYPE` statement rolls back.

Metadata for all tables and columns that use an altered type are updated for the new type definition so that data can be stored in them in the new format. Existing data can be converted to the new format either all at once or piecemeal as it is updated. In either case, data is always presented in the new type definition, even if it is still stored in the format of the older one.

The following example adds the attribute `address` and removes the attribute `phone` from the `person_typ`.

```
ALTER TYPE person_typ
  ADD ATTRIBUTE (address VARCHAR2(100)),
  DROP ATTRIBUTE phone CASCADE NOT INCLUDING TABLE DATA;
```

The option `CASCADE` propagates the type change to dependent types and tables. The option `NOT INCLUDING TABLE DATA` asks Oracle to leave column data as is and does not change the type version.

Object Tables

An object table is a special table in which each row represents an object. Example 4 creates an object table for `person_typ` objects.

Example 4: Creating the `person_obj_table` Object Table

```
CREATE TABLE person_obj_table OF person_typ;
```

We can view this table in two ways:

- As a [single-column table](#) in which each row is a `person_typ` object, allowing us to perform object-oriented operations
- As a [multi-column table](#) in which each attribute of the object type `person_typ`; such as `idno`, `name`, and `phone`; occupies a column, allowing you to perform relational operations

The following example illustrates several operations on an object table.

Example 5: Operations on the `person_obj_table` Object Table

```
INSERT INTO person_obj_table VALUES (  
    person_typ(101, 'John', 'Smith', 'oop@gmail.com', '091234455')  
);
```

```
SELECT VALUE(p) FROM person_obj_table p  
    WHERE p.last_name = 'Smith';
```

```
DECLARE  
    person person_typ;  
BEGIN -- PL/SQL block for selecting a person and displaying details  
    SELECT VALUE(p) INTO person FROM person_obj_table p  
        WHERE p.idno = 101;  
  
    person.display_details();  
END;  
/
```

The first SQL statement in Example 5 inserts a `person_typ` object into `person_obj_table`, treating `person_table` as a multi-column table. The second SQL statement selects from `person_obj_table` as a single-column table, using the `VALUE` function to return rows as object instances. See "VALUE" for information on the `VALUE` function.

References

A REF is a [logical pointer](#) to a row object that is constructed from the object identifier (OID) of the referenced object and is an Oracle built-in datatype. REFs and collections of REFs model associations among objects, particularly many-to-one relationships, thus reducing the need for

foreign keys. REFs provide an easy mechanism for navigating between objects. You can use the dot notation to follow the pointers. Oracle does joins for you when needed, and in some cases can avoid doing joins.

We can use a REF to examine or update the object it refers to. You can also use a REF to obtain the object it refers to. You can change a REF so that it points to a different object of the same object type hierarchy or assign it a null value.

The following example illustrates a simple use of a REF.

Example 6: Using a REF to the emp_person_typ Object.

```
CREATE TYPE emp_person_typ AS OBJECT (  
    name      VARCHAR2(30),  
    manager   REF emp_person_typ  
);  
/  
  
CREATE TABLE emp_person_obj_table OF emp_person_typ;  
  
INSERT INTO emp_person_obj_table VALUES (  
    emp_person_typ ('John Smith', NULL));  
  
INSERT INTO emp_person_obj_table  
    SELECT emp_person_typ ('Bob Jones', REF(e))  
    FROM emp_person_obj_table e  
    WHERE e.name = 'John Smith';
```

More readings

- Collection types: For modeling multi-valued attributes and many to many relationships, Oracle supports two collection datatypes: varrays and nested tables.
- Inheritance (subtypes): PL/SQL allows creating object from the existing base objects. To implement inheritance, the base objects should be declared as NOT FINAL. The default is FINAL.
- Abstract objects: We cannot use an abstract object as it is. We will have to create a subtype or child type of such objects to use its functionalities.

Exercises

1. Write a PL/SQL code to create a class for a "Person" with attributes such as name, age, and address.
2. Create an object **p1** of type Person with the following attribute values: ('John Doe', 30, '123 Main Street') and display its attributes to the screen.

3. Write a PL/SQL code to implement methods in the "Person" class to display the details and update the age.
4. Write a PL/SQL code to implement a method to calculate the annual bonus based on the salary in the "Employee" class, given:

```
salary < 50000:  bonus_percentage := 0.1;
salary < 100000: bonus_percentage := 0.15;
otherwise:      bonus_percentage := 0.2;
bonus_amount = salary * bonus_percentage;
```

5. Write a PL/SQL code to create a "Shape" class with attributes like length and width and implement a method to calculate the area.
6. Write a PL/SQL code that implements a method in the "Shape" class to determine whether the shape is square.
7. **[Extra]** Write a PL/SQL code to create a "Manager" subclass inheriting from the "Employee" class, and add an attribute to store the number of employees managed. A subclasses "Manager" subclass that inherits from the "Employee" class and adds an additional attribute to store the number of employees managed by the manager.